

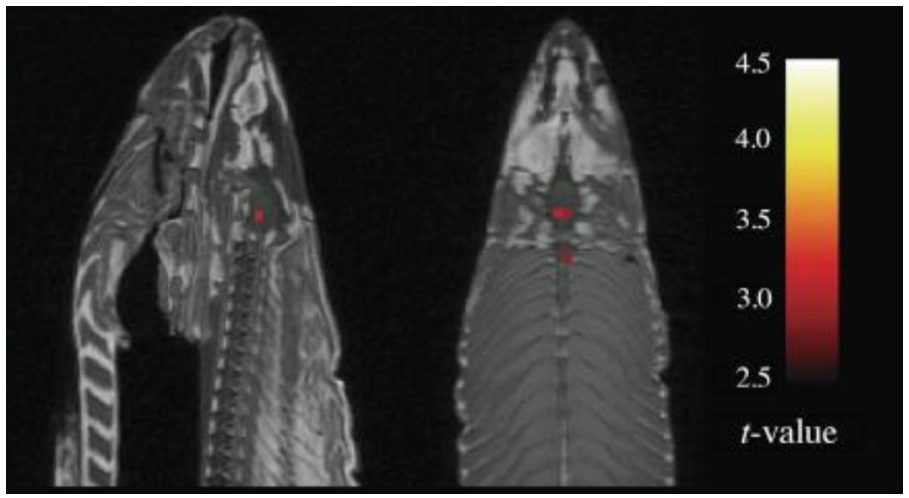
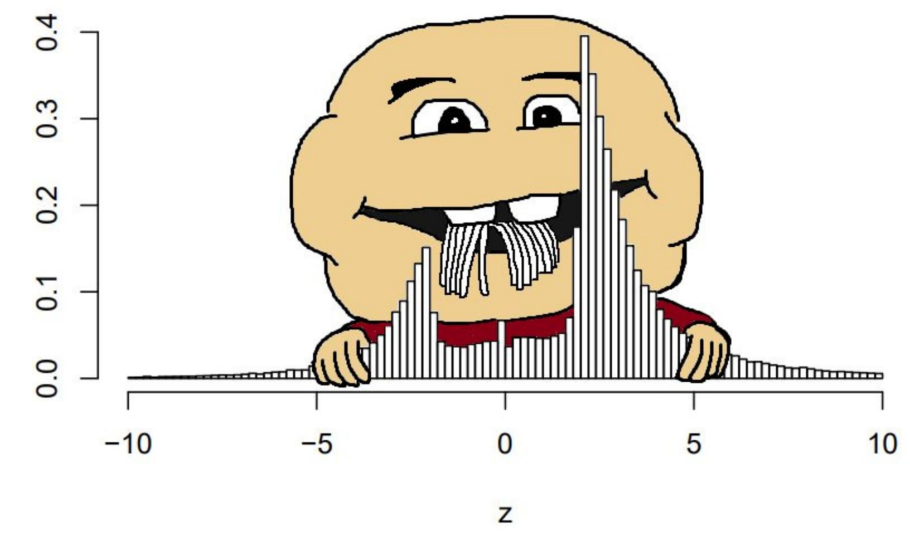
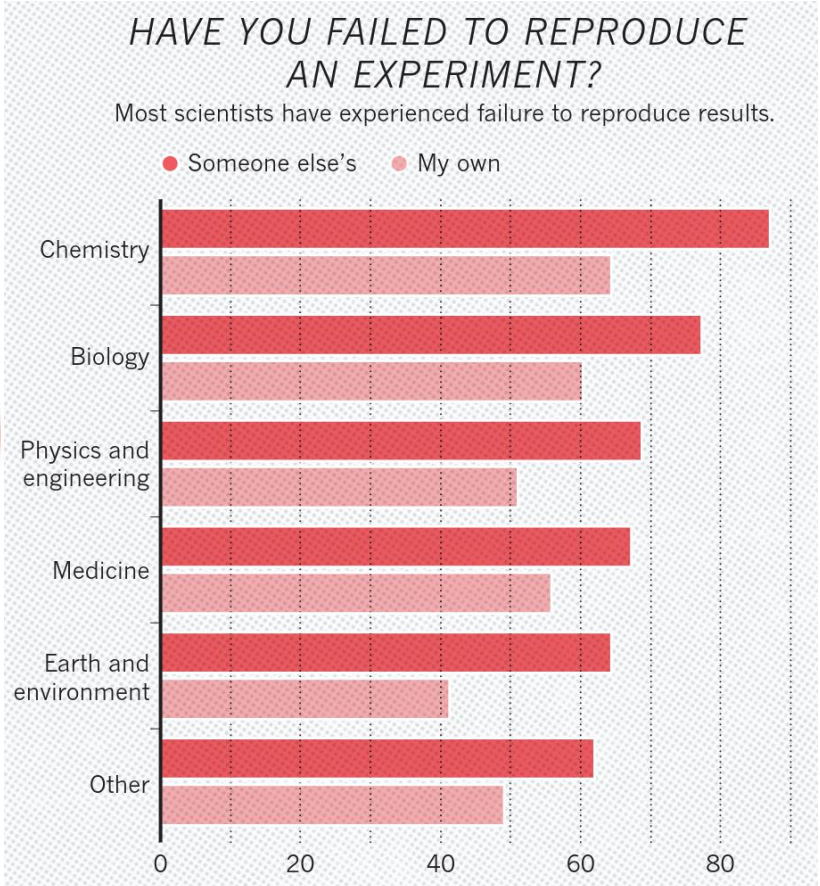
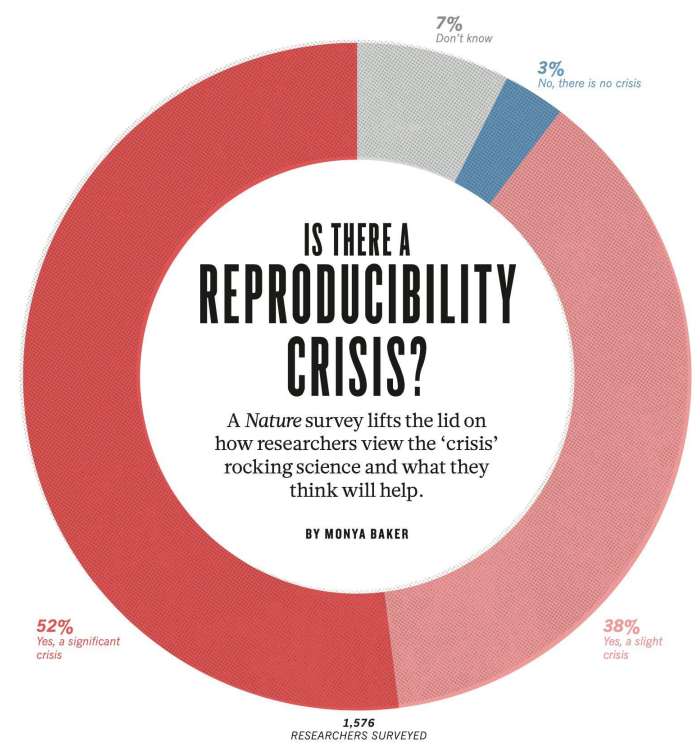


Оркестрация вычислений в биоинформатике

Лекция 9. Воспроизводимые ML-
пайплайны. mlflow. Простой фронтенд в
streamlit.

Воспроизводимость

По сути, весь наш курс посвящён воспроизводимости. Тем не менее,



Воспроизводимость

Различается в зависимости от того, как и с кем вы работаете.
Рассмотрим худшие случаи

	Индивидуально	С кем-то	С кем-то в едином окружении
Repeatability			
Reproducibility			
Replicability			

Воспроизводимость

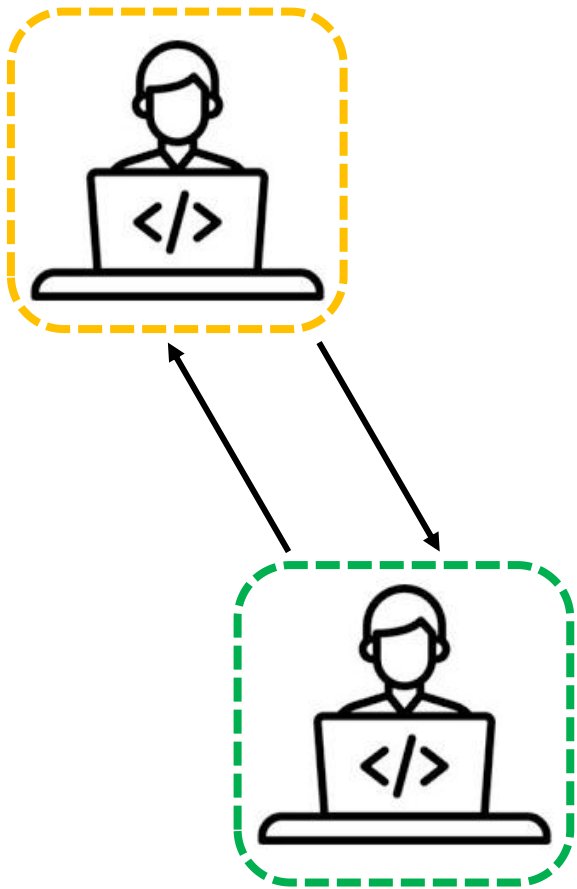
Различается в зависимости от того, как и с кем вы работаете.



	Индивидуально	С кем-то	С кем-то в едином окружении
Repeatability	?		
Reproducibility	—		
Replicability	—		

Воспроизводимость

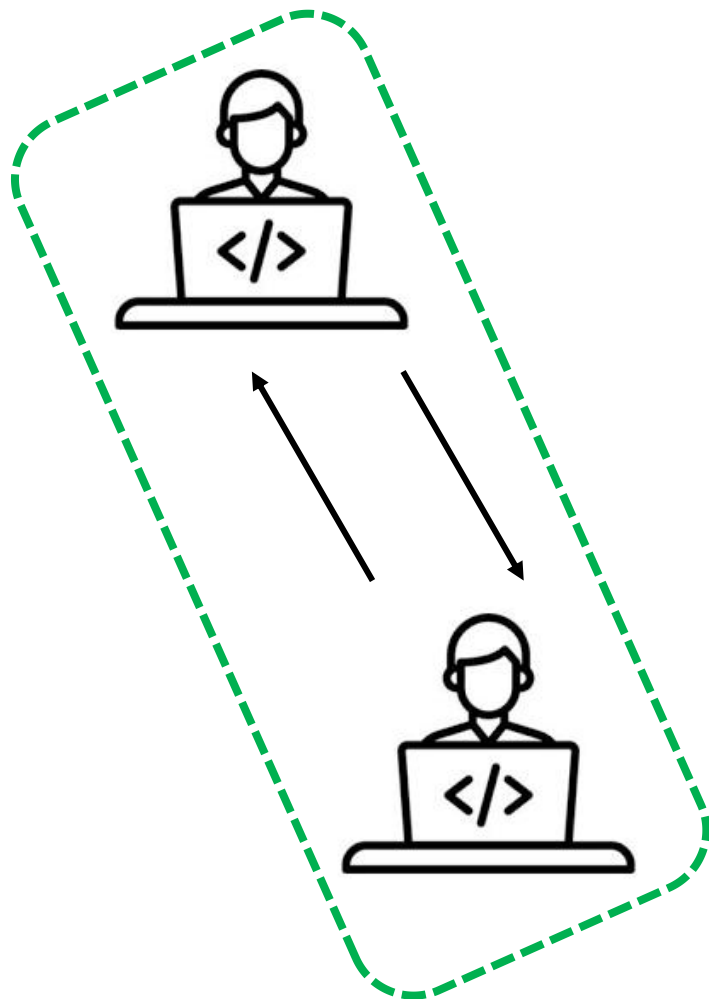
Различается в зависимости от того, как и с кем вы работаете.



	Индивидуально	С кем-то	С кем-то в едином окружении
Repeatability	?	+	
Reproducibility	-	?	
Replicability	-	-	

Воспроизводимость

Различается в зависимости от того, как и с кем вы работаете.



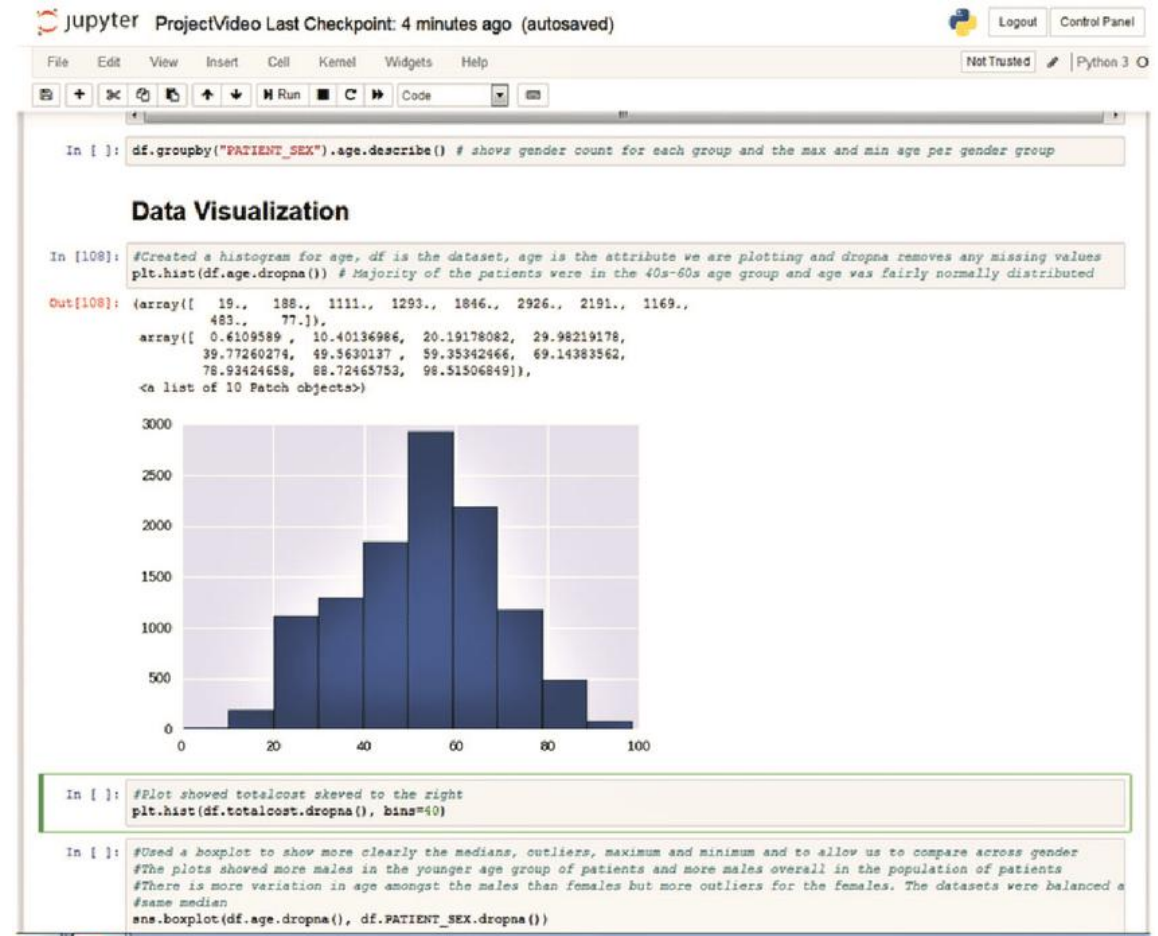
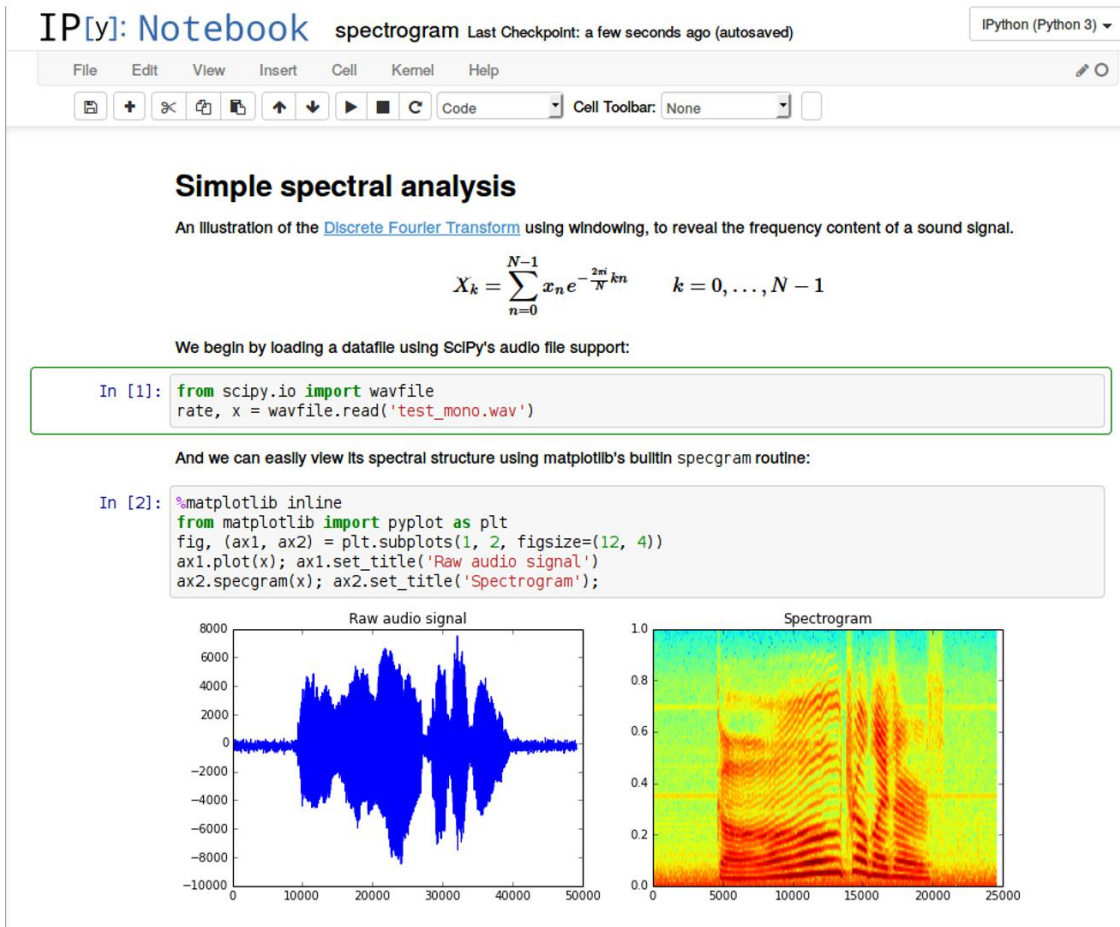
	Индивидуально	С кем-то	С кем-то в едином окружении
Repeatability	?	+	+
Reproducibility	-	?	+
Replicability	-	-	?

Что надо контролировать?

1. Среда: venv или контейнер с вычислениями
2. Данные: источники, обработки, качество/полнота
3. Исполняемость кода
4. **Эксперимент**

Как выглядит эксперимент ML/DS

Хочется, чтоб он всегда выглядел так (часто даже 1 ноутбук = 1 задача)



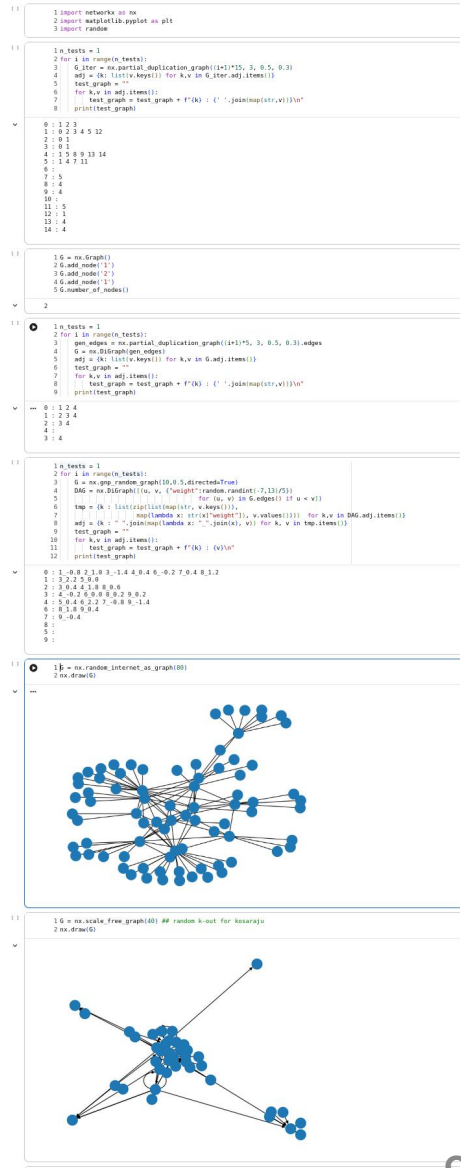
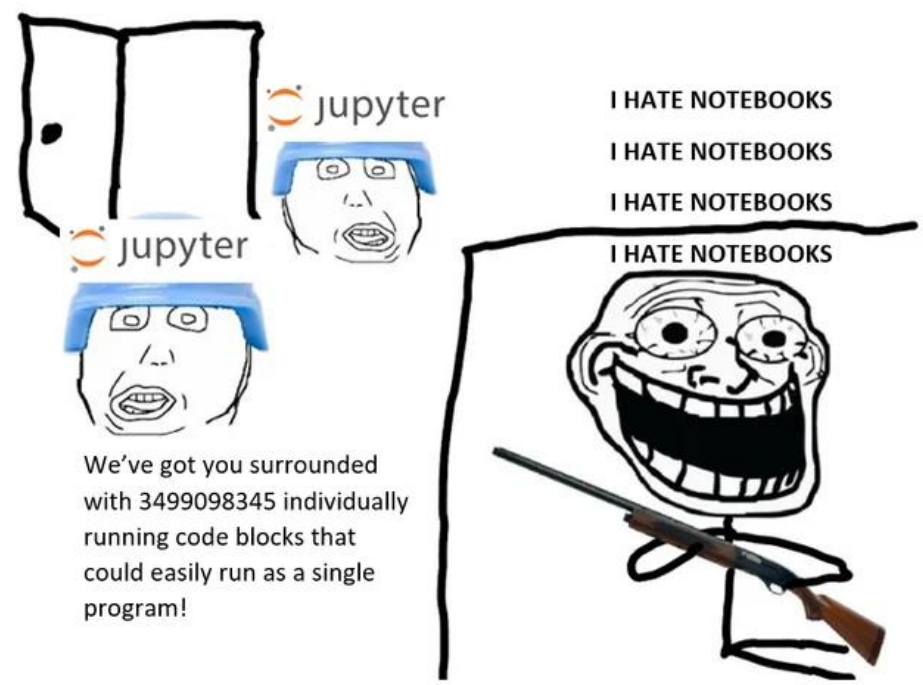
Как выглядит эксперимент ML/DS

В реальности всё куда более печально.

```
1 header_dt = {}
2
3 with open("save_res_err.txt", 'r') as fin:
4     header = fin.readline().strip().strip('.')
5     errors = []
6     line = fin.readline().strip()
7     while line != "":
8         if line.startswith("-----"):
9
10             header_dt[header] = list(set(errors))
11             header = line.strip().strip('.')
12             errors = []
13             if (line.find(": ") != -1 and
14                 "INFO" not in line and
15                 "ERROR" not in line and
16                 "# type" not in line and
17                 "FileNotFound" not in line) or line == "EOFError":
18                 errors.append(line)
19             line = fin.readline().strip()
20             header_dt[header] = errors
21
22 header_dt = {key: [error for error in value if "airflow" not in error] for key, value in header_dt.items()}
```

```
1 for key in header_dt:
2     if header_dt[key] == []:
3         header_dt[key] = ["airflow.exceptions.AirflowException: Task received SIGTERM signal"]
4     if len(header_dt[key]) == 3:
5         header_dt[key] = ["ValueError: time data 'NA' does not match format '%Y-%m-%d'"]
6     if len(header_dt[key]) == 2:
7         if 'ftplib' in header_dt[key][0] or 'ftplib' in header_dt[key][1]:
8             header_dt[key] = [' && '.join(header_dt[key])]
```

```
1 s = ""
2 manual_2024-07-12T16:21:39.561428+00:00
3 manual_2024-07-12T16:21:41.682459+00:00
4 manual_2024-07-12T16:21:48.943040+00:00
5 manual_2024-07-12T16:21:50.000000+00:00
```



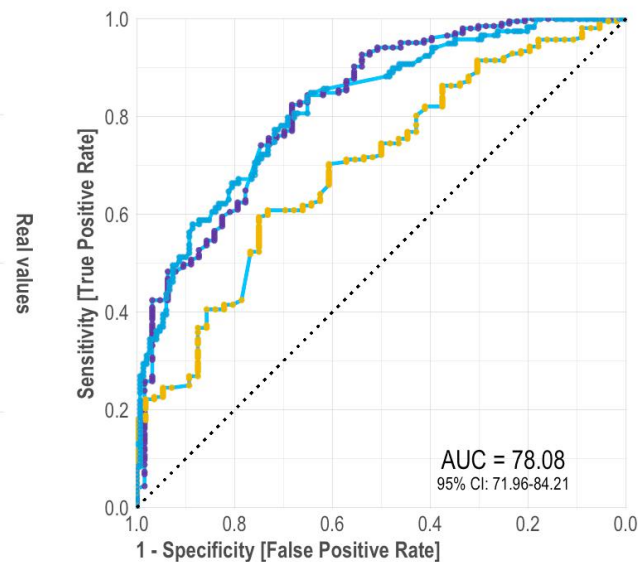
Гигиена jupyter

Тетрадь это основной лэбжурнал экспериментов. В нём стоит прописывать всё, что вы делаете (в т.ч., неправильное!!)

Самые дорогие выводы -- те, которые нельзя воспроизвести.

	n_1	n_2	n_3
n_1 (63)	35 13.1%T (55.6%)	8 3.0%T (12.7%)	20 7.5%T (31.7%)
n_2 (56)	15 5.6%T (26.8%)	17 6.3%T (30.4%)	24 9.0%T (42.9%)
n_3 (149)	11 4.1%T (7.4%)	14 5.2%T (9.4%)	124 46.3%T (83.2%)

XGBoost_1_AutoML_20200802_153027



• 69.66% | n_2 • 82.12% | n_3 • 82.47% | n_1



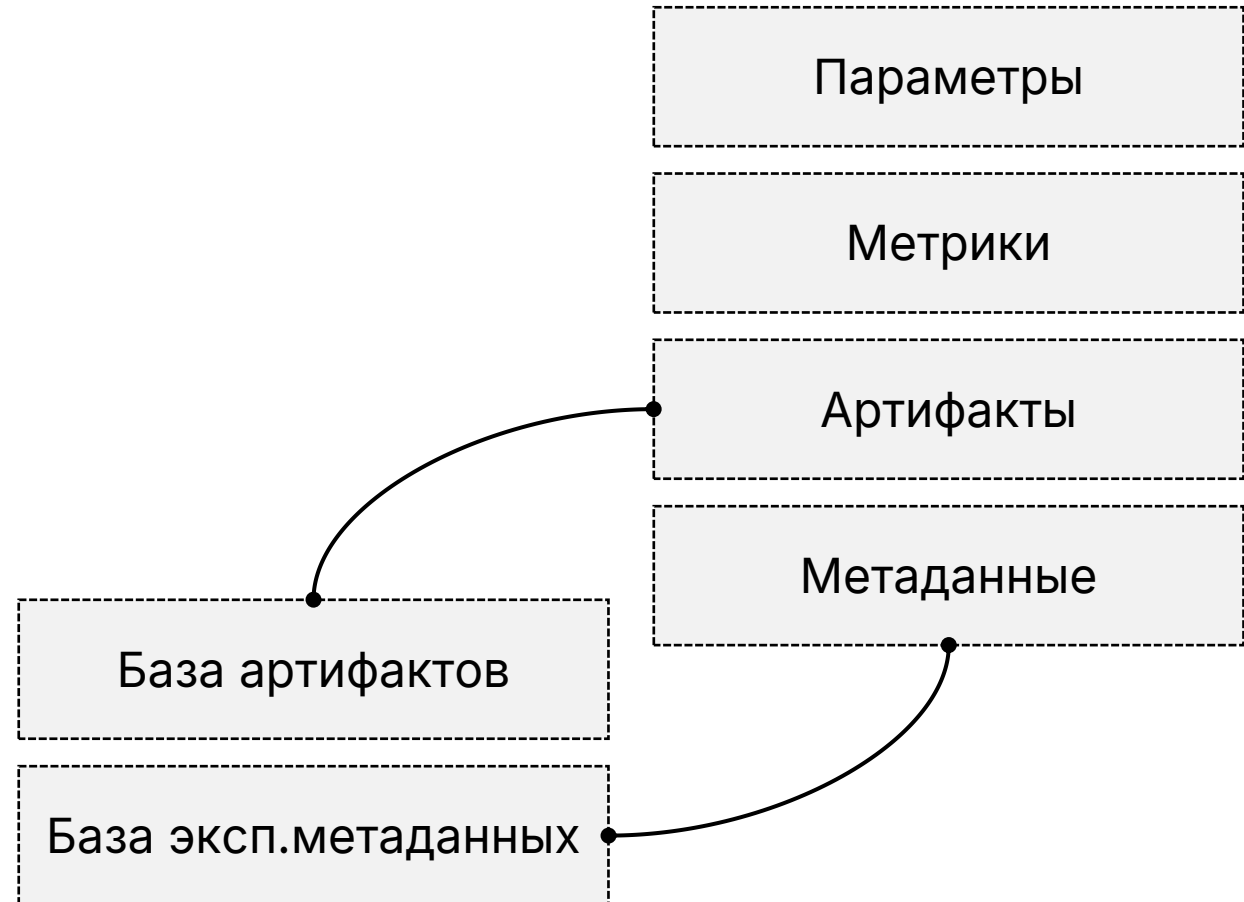
артифакты эксперимента

Решение: mlflow™

Эксперимент

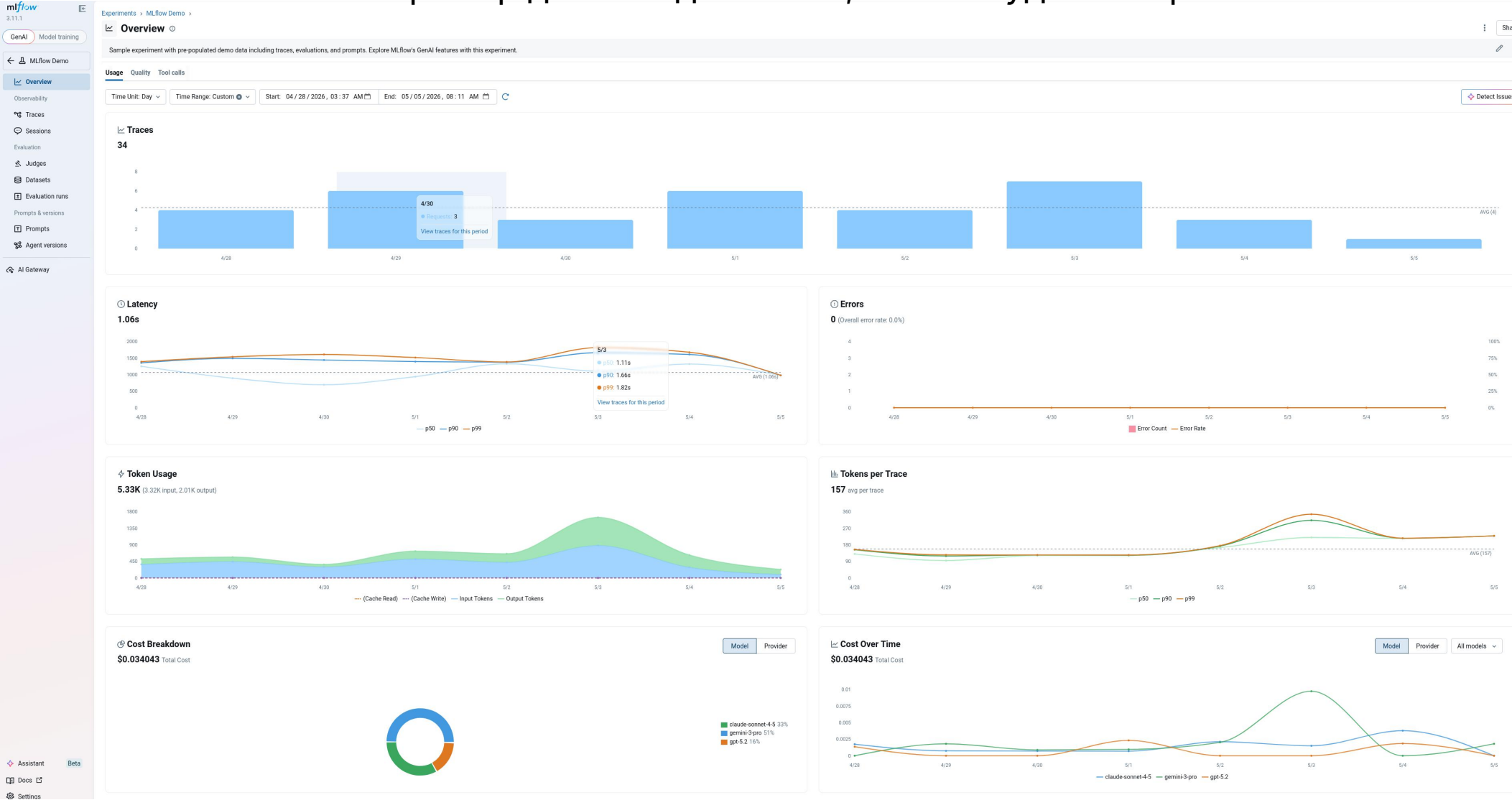


mlflow run



mlflow UI

Пример для вкладки GenAI, но нас будет интересовать model training



Как ЭТИМ пользоваться?

Можно через UI, но мы пойдём через python.

1. Нужно создать локальную БД для артефактов (можно через СУБД)

```
mlflow.set_tracking_uri("./mlruns")
```

2. В пределах эксперимента логируем всё, что можем

```
mlflow.set_experiment("experiment-name")
with mlflow.start_run(run_name=model_name):
    mlflow.log_param("model_type", model_name)
    for param_name, param_value in params.items():
        mlflow.log_param(param_name, param_value)

    mlflow.log_param("train_samples", len(X_train))
    mlflow.log_param("test_samples", len(X_test))
    mlflow.log_param("n_features", len(feature_names))
```

информация о
данных и параметрах
модели

Как ЭТИМ пользоваться?

1. Вообще всё

```
with mlflow.start_run(run_name=model_name):  
    ...  
    model.fit(X_train, y_train)  
    metrics, y_pred = evaluate_model(model, X_test, y_test)  
    for metric_name, metric_value in metrics.items():  
        mlflow.log_metric(metric_name, metric_value)  
  
    cm_fig = plot_confusion_matrix(y_test, y_pred, model_name)  
    mlflow.log_figure(cm_fig, "confusion_matrix.png")  
    plt.close(cm_fig)  
  
    mlflow.sklearn.log_model(model, name=model_name)  
    report = classification_report(  
        y_test, y_pred, target_names=["Failed", "Success"]  
    )  
    mlflow.log_text(report, "classification_report.txt")
```

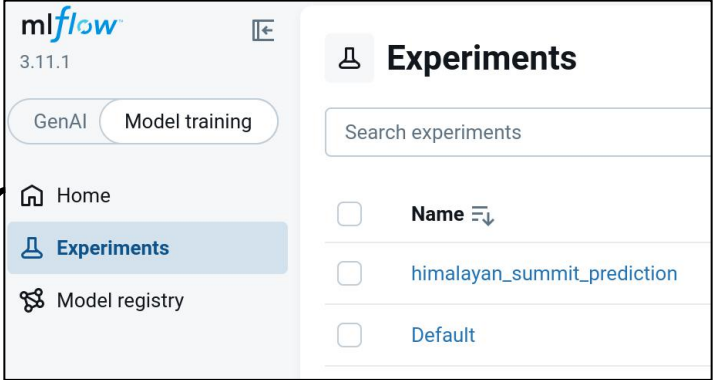
значения метрик

графики

саму модель!

текст

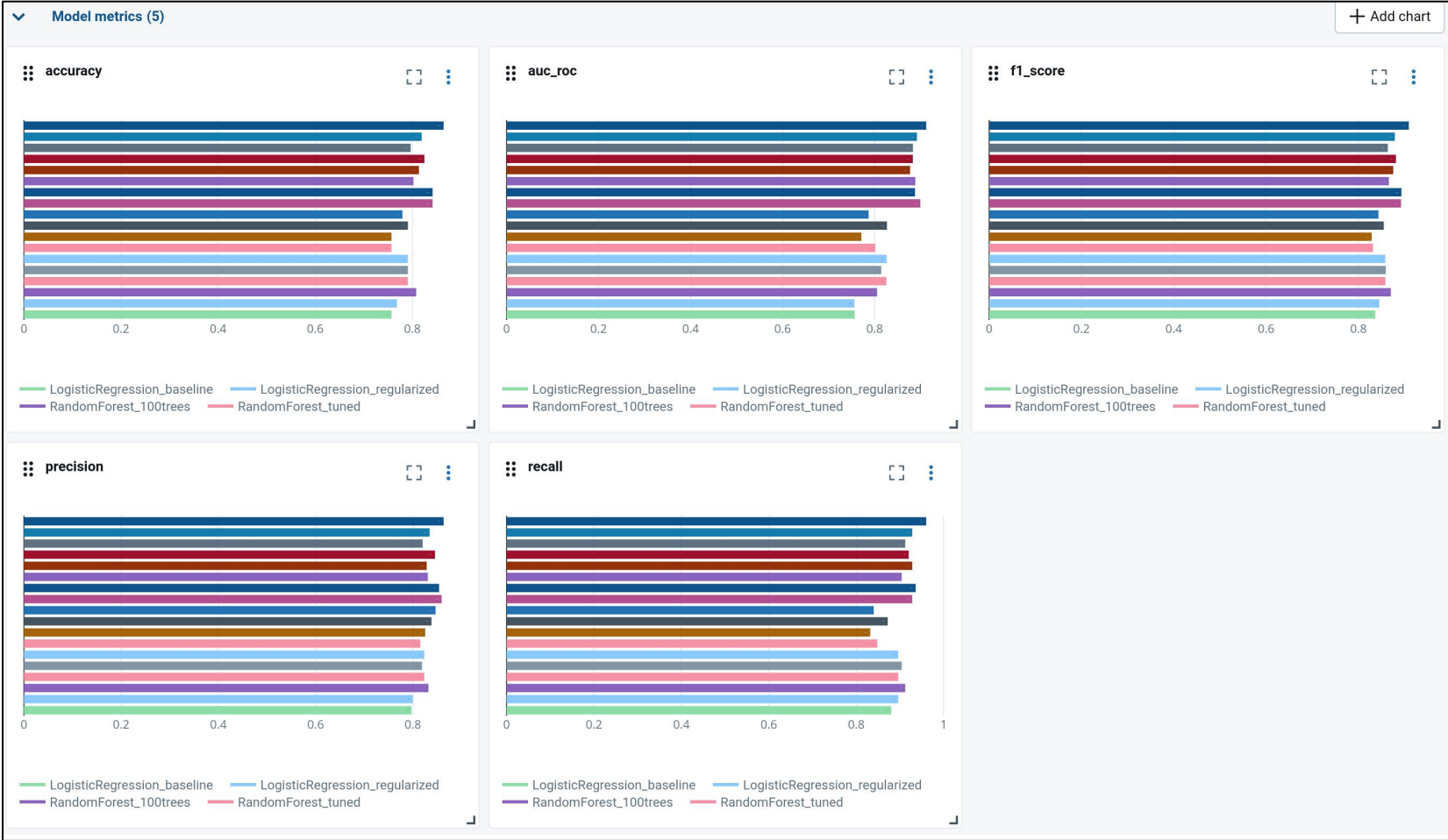
Плоды трудов



Runs

metrics.rmse < 1 and params.model = "tree"

☐	Run Name	Created	Data
☐	MLP_64_32	12 minutes ago	-
☐	SVM_RBF	12 minutes ago	-
☐	KNN_7neighbors	12 minutes ago	-
☐	ExtraTrees	12 minutes ago	-
☐	GradientBoosting_tuned	13 minutes ago	-
☐	GradientBoosting_default	13 minutes ago	-
☐	RandomForest_tuned	13 minutes ago	-
☐	RandomForest_100trees	13 minutes ago	-
☐	LogisticRegression_regul...	13 minutes ago	-
☐	LogisticRegression_basel...	13 minutes ago	-



```
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

Еще пример

Перебор гиперпараметров для RF



После экспериментов

Надо выбрать лучшую модель.

```
def find_best_model():
    experiments = [...]
    all_runs = []
    for exp_name in experiments:
        runs = mlflow.search_runs(experiment_names=[exp_name])
    all_runs_df = pd.concat(all_runs, ignore_index=True)
    best_run = all_runs_df.sort_values("metrics.f1_score", ascending=False).iloc[0]
    return best_run

def load(run_id, artifact_name):
    return mlflow.sklearn.load_model(f"runs:{run_id}/{artifact_name}")

def register_model(run_id, artifact_name, model_name="HimalayanSummitPredictor"):
    model_uri = f"runs:{run_id}/{artifact_name}"
    result = mlflow.register_model(model_uri, model_name)
    print(f"Model registered successfully!")
    return result
```

После экспериментов

Кроме того, в бигтехе лучшую модель ещё нужно выкатить. mlflow позволяет создать сервер модели, на который можно отправлять API-запросы:

```
mlflow models serve -m "models:/HimalayanSummitPredictor/latest" \
                    -p 5001 --no-conda
```

```
def predict_expedition(expedition_data: dict):
    url = "http://127.0.0.1:5001/invocations"
    response = requests.post(url, json=expedition_data)
    return response.json()

everest_spring = {
    "dataframe_split": {
        "columns": [% all dataset columns %],
        "data": [[Sample 1], [Sample2], ...]
    }
}
```

```
python3 serve_demo.py
Everest Spring
Expedition:
{'predictions': [1]}
Solo Winter 8000er:
{'predictions': [1]}
```

Что дальше?

Научить модель -- половина дела. Хорошо уметь её питчить. Как показать всем, что ваша модель хорошая? Как рассказать историю?

Что дальше?

Научить модель -- половина дела. Хорошо уметь её питчить. Как показать всем, что ваша модель хорошая? Как рассказать историю?

1. EDA - набор картинок и стат.тестов с моделью в ноутбуке

Что дальше?

Научить модель -- половина дела. Хорошо уметь её питчить. Как показать всем, что ваша модель хорошая? Как рассказать историю?

1. ~~EDA набор картинок и стат.тестов с моделью в ноутбуке~~

Мы только отошли от `ipynb`-тетрадок! Есть ли альтернативы?

Что дальше?

Научить модель -- половина дела. Хорошо уметь её питчить. Как показать всем, что ваша модель хорошая? Как рассказать историю?

1. ~~EDA набор картинок и стат.тестов с моделью в ноутбуке~~

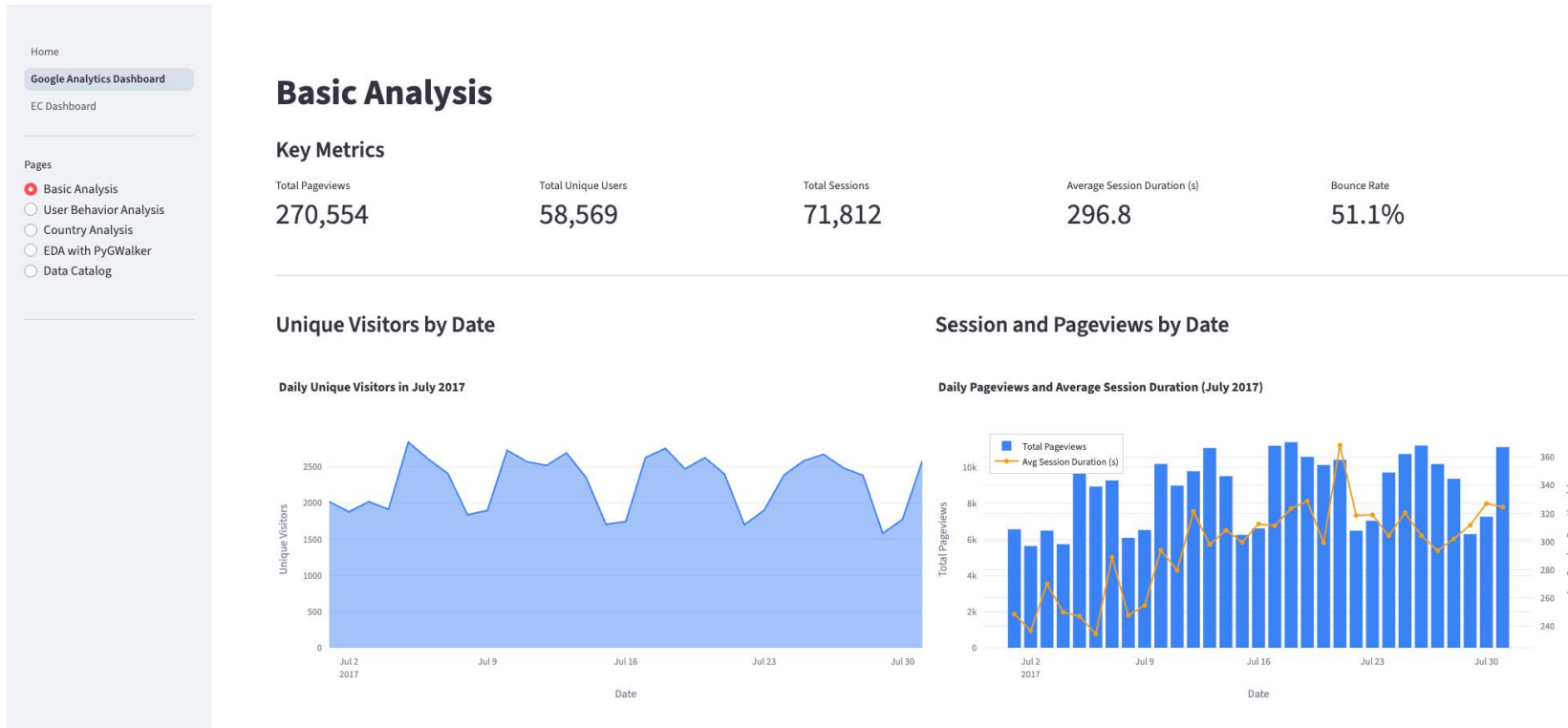
Мы только отошли от ipynb-тетрадок! Есть ли альтернативы?

2. Дашборды

3. Интерактивные приложения (streamlit)

Streamlit

Пакет для веб-приложений в питоне, переносимых и понятных конечному пользователю. Можно деплоить



Streamlit - очень большая система

Основные моменты:

- код = приложение
- можно несколько страниц
- куча функций для любого функционала
- интеграция с библиотеками

`st.columns`

`st.metric`

`st.plotly_chart`

`st.dataframe`

`st.sidebar`

`st.title`

`st.write`

`st.button`

`st.markdown`

`st.pyplot`

`st.spinner`

`st.warning`

Streamlit - пример

См. в листинге код для приложения

